

Exclude Properties from Model Binding in ASP.NET Core Web API

Back to: [ASP.NET Core Web API Tutorials](#)

How to Exclude Properties from Model Binding in ASP.NET Core Web API

In this article, I will discuss **how to Exclude Properties from Model Binding in ASP.NET Core Web API** Applications with Examples. Please read our previous article discussing How to Implement [Content Negotiation in ASP.NET Core Web API](#) Application with Examples.

In ASP.NET Core Web API, **Model Binding** automatically maps incoming request data (such as route values, query strings, headers, or JSON bodies) to your model properties and action parameters. This makes APIs easy to use, as developers can work directly with strongly-typed C# objects instead of manually parsing HTTP request data.

However, not all model properties should be exposed to or modifiable by clients. Some properties are **Calculated Internally**, some are **Sensitive Business Values**, and others are **System-Generated** (like unique identifiers or internal pricing logic). In such cases, it becomes essential to **control which properties participate in model binding** to ensure **data integrity**, **security**, and **clean API design**.

Why Exclude Properties from Model Binding?

There are several practical reasons for excluding specific properties from model binding:

1. **Security:** Prevents over-posting attacks, where malicious users attempt to modify sensitive data (like Cost, Discounts, or Internal IDs).
2. **Data Integrity:** Ensures internal business rules (like cost calculations or SKU generation) are always computed by the server, not the client.
3. **Clean API Design:** Keeps public contracts simple and focused on what the client actually needs to send or receive.
4. **Performance:** Reduces unnecessary payloads, minimizing serialization and validation overhead.

By carefully controlling model binding, we can keep the public API clean while protecting the internal data flow of our system.

Real-World Example: Product Inventory Management System

Imagine a **Product Inventory Management System** for an e-commerce platform. Every product in this system contains several key pieces of information:

- **ProductId**: A unique identifier automatically generated by the system.
- **SKU**: A system-generated **Stock Keeping Unit**, unique for each product, and immutable once created.
- **SellingPrice**: The price set by the admin or retailer.
- **CostPrice**: The internal cost is calculated based on the supplier discount.
- **SupplierDiscount**: A sensitive internal metric, not visible to the client.
- **IsActive**: Indicates whether the product is currently active in the catalog.

When a new product is created, only basic details like Name, Category, and SellingPrice are provided by the client. The API internally calculates CostPrice, generates a unique SKU, and applies system-defined discounts, all automatically. This is a perfect real-world example where **some properties must be excluded** from model binding and managed internally.

Defining the Product Model

This is the core entity representing a product in the system. It contains **all fields**, both client-visible and internal. The Product class below defines every piece of product data the system uses internally. However, not all of them should be exposed via the API. Properties like ProductId, CostPrice, and SupplierDiscount are strictly managed by the server and not accepted from clients.

```
namespace ModelBindingDemo.Models
{
    public class Product
    {
        public int ProductId { get; set; }           // Generated by system
        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public string SKU { get; set; } = string.Empty; // Generated automatically
        public decimal SellingPrice { get; set; }      // Set by admin/client
        public decimal CostPrice { get; set; }         // Calculated internally
        public decimal SupplierDiscount { get; set; }  // Internal business logic
        public bool IsActive { get; set; } = true;
    }
}
```

Note: In Real-time, the Product entity represents the database-level model or business domain object. It's not meant to be directly exposed in the API; instead, we will use **DTOs** for safe data exchange.

Creating the Product Controller

Now, let's create a simple controller to simulate product management APIs.

```
using Microsoft.AspNetCore.Mvc;
using ModelBindingDemo.Models;

namespace ModelBindingDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        // In-memory data store
        private static readonly List<Product> _products = new()
        {
            new Product { ProductId = 1, Name = "Smartphone", Category =
"Electronics", SKU = "ELEC-001", SellingPrice = 29999, SupplierDiscount = 0.10M,
CostPrice = 27000 },
            new Product { ProductId = 2, Name = "Running Shoes", Category =
"Sports", SKU = "SPRT-101", SellingPrice = 3999, SupplierDiscount = 0.15M,
CostPrice = 3400 }
        };

        // GET: /api/products
        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAllProducts()
        {
            return Ok(_products);
        }

        // POST: /api/products
        [HttpPost]
        public ActionResult<Product> AddProduct(Product product)
        {
            if (product == null)
                return BadRequest("Invalid product data.");

            // Simulate server-side generation logic
            product.ProductId = _products.Count + 1;

            // The client can still send SupplierDiscount and CostPrice
            // Ideally, these should not be accepted from the request.
            // For demonstration, let's still recalculate CostPrice.
            product.CostPrice = product.SellingPrice * (1 -
product.SupplierDiscount);
        }
    }
}
```

```

        // SKU Generation Logic (system-generated)
        product.SKU = GenerateSKU(product.Category, product.ProductId);

        _products.Add(product);

        return CreatedAtAction(nameof(GetAllProducts), new { id =
product.ProductId }, product);
    }

    // Generates a unique SKU (Stock Keeping Unit) for a product based on its
category and ID
    private string GenerateSKU(string category, int id)
    {
        // If the category name has 4 or more characters, take the first 4
letters and convert them to uppercase.
        // Otherwise, take the entire category name and convert it to
uppercase.
        // Example: "Electronics" → "ELEC", "Toy" → "TOY"
        var prefix = category.Length >= 4
            ? category.Substring(0, 4).ToUpper()
            : category.ToUpper();

        // Combine the category prefix with the product ID.
        // {id:D4} means format the integer ID as a 4-digit number with
leading zeros if needed.
        // Example: id = 7 → "0007", id = 45 → "0045"
        // Final SKU example: "ELEC-0007"
        return $"{prefix}-{id:D4}";
    }
}
}

```

Testing Behavior

- When you send a **GET** request → **/api/products**, you will receive all products, including sensitive fields (CostPrice, StockCount, ProductId).
- When you send a **POST** request → **/api/products**, if the client includes these sensitive fields, they'll be **overwritten** in the controller logic.

What's the Problem?

When clients send data to the API, they can include sensitive properties like SupplierDiscount or CostPrice in the request body. This exposes internal logic and creates opportunities for **over-posting** and **data tampering**. We will now fix that by explicitly **excluding** such fields from model binding.

How to Exclude Properties from Model Binding

We can restrict specific properties from being bound or serialized by using the **[JsonIgnore] attribute** or DTOs.

Using [JsonIgnore] Attribute

ASP.NET Core provides the [JsonIgnore] attribute (from the System.Text.Json.Serialization namespace), which allows us to **exclude properties** from both serialization (response) and deserialization (request). This means:

- If a property is marked [JsonIgnore], the client cannot send it (it won't bind).
- It also won't appear in the response JSON when the server sends data back.

Updated Product Model

```
using System.Text.Json.Serialization;
namespace ModelBindingDemo.Models
{
    public class Product
    {
        [JsonIgnore]
        public int ProductId { get; set; }

        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;

        public string SKU { get; set; } = string.Empty; // Externally visible,
immutable

        public decimal SellingPrice { get; set; }

        [JsonIgnore]
        public decimal CostPrice { get; set; }

        [JsonIgnore]
        public decimal SupplierDiscount { get; set; }

        public bool IsActive { get; set; } = true;
    }
}
```

Code Explanation:

This approach ensures:

- Clients cannot modify ProductId, CostPrice, or SupplierDiscount.

- These properties remain internal, managed only by the backend.

However, this approach is **static**; it hides fields globally for all operations. If you need flexibility (e.g., show SKU in responses but not accept it in requests), you should use **DTOs**.

Example Behavior

POST Request Sent by Client:

```
{
  "productId": 100,
  "name": "Wireless Headphones",
  "category": "Audio",
  "sku": "AUD-999",
  "sellingPrice": 4999,
  "costPrice": 3000,
  "supplierDiscount": 0.25,
  "isActive": true
}
```

Actual Bound Object (Server-Side):

```
{
  "productId": 0,
  "name": "Wireless Headphones",
  "category": "Audio",
  "sku": "AUD-999",
  "sellingPrice": 4999,
  "costPrice": 3749.25,
  "supplierDiscount": 0,
  "isActive": true
}
```

When to Use [JsonIgnore]

Use [JsonIgnore] when you need to **permanently exclude** sensitive or system-managed fields from client input and output. This works perfectly for internal values such as computed prices, margins, or IDs.

Using Data Transfer Objects (DTOs)

DTOs (Data Transfer Objects) are purpose-built models for API interaction. They let us control what the client **sends** and **receives**, independently of the primary entity. We'll create:

- **CreateProductDTO** for accepting input from the client (POST /api/products).

- The **ProductResponseDTO** is used for returning data to the client (GET /api/products).

This separation ensures precise control over data flow and security.

CreateProductDTO (for POST requests)

```
namespace ModelBindingDemo.Models
{
    public class CreateProductDTO
    {
        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public decimal SellingPrice { get; set; }
        public bool IsActive { get; set; } = true;
    }
}
```

Explanation

This DTO includes only the properties that the **client is allowed to send** when creating a product. No ProductId, SKU, SupplierDiscount, or CostPrice; all those are **system-generated or internal**. This ensures:

- Simplicity for the client.
- Full control for the backend logic.

ProductResponseDTO (for GET responses)

```
namespace ModelBindingDemo.Models
{
    public class ProductResponseDTO
    {
        public int ProductId { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public string SKU { get; set; } = string.Empty;
        public decimal SellingPrice { get; set; }
        public bool IsActive { get; set; } = true;
    }
}
```

Explanation

This DTO defines the shape of data that is returned to the client when a product is fetched. Sensitive fields like CostPrice and SupplierDiscount are excluded, while safe, user-facing details like SKU and SellingPrice remain visible.

Modify Product Entity

Please modify the Product entity to remove the JsonIgnore Attribute.

```
namespace ModelBindingDemo.Models
{
    public class Product
    {
        public int ProductId { get; set; }
        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public string SKU { get; set; } = string.Empty; // Externally visible,
immutable
        public decimal SellingPrice { get; set; }
        public decimal CostPrice { get; set; }
        public decimal SupplierDiscount { get; set; }
        public bool IsActive { get; set; } = true;
    }
}
```

Modifying Controller

Now, let's build the modified ProductsController that uses these DTOs.

```
using Microsoft.AspNetCore.Mvc;
using ModelBindingDemo.Models;

namespace ModelBindingDemo.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private static readonly List<Product> _products = new()
        {
            new Product { ProductId = 1, Name = "Smartphone", Category =
"Electronics", SKU = "ELEC-001", SellingPrice = 29999, SupplierDiscount = 0.10M,
CostPrice = 27000 },
            new Product { ProductId = 2, Name = "Running Shoes", Category =
"Sports", SKU = "SPRT-101", SellingPrice = 3999, SupplierDiscount = 0.15M,
CostPrice = 3400 }
        };

        // GET: /api/products
        [HttpGet]
        public ActionResult<IEnumerable<ProductResponseDTO>> GetProducts()
```

```

{
    var result = _products.Select(p => new ProductResponseDTO
    {
        ProductId = p.ProductId,
        Name = p.Name,
        Category = p.Category,
        SKU = p.SKU,
        SellingPrice = p.SellingPrice,
        IsActive = p.IsActive
    }).ToList();

    return Ok(result);
}

// POST: /api/products
[HttpPost]
public ActionResult<ProductResponseDTO> AddProduct(CreateProductDTO dto)
{
    if (dto == null)
        return BadRequest("Invalid product data.");

    // Create a new Product entity
    var newProduct = new Product
    {
        ProductId = _products.Count + 1,
        Name = dto.Name,
        Category = dto.Category,
        SellingPrice = dto.SellingPrice,
        SupplierDiscount = 0.12M // Default system discount
    };

    // Internal calculation
    newProduct.CostPrice = newProduct.SellingPrice * (1 -
newProduct.SupplierDiscount);

    // SKU generation logic
    newProduct.SKU = GenerateSKU(newProduct.Category,
newProduct.ProductId);

    _products.Add(newProduct);

    // Map to response DTO
    var response = new ProductResponseDTO
    {
        ProductId = newProduct.ProductId,
        Name = newProduct.Name,
        Category = newProduct.Category,

```

```

        SKU = newProduct.SKU,
        SellingPrice = newProduct.SellingPrice,
        IsActive = newProduct.IsActive
    };

    return CreatedAtAction(nameof(GetProducts), new { id =
newProduct.ProductId }, response);
}

// SKU Generation Logic
private string GenerateSKU(string category, int id)
{
    var prefix = category.Length >= 4 ? category.Substring(0,
4).ToUpper() : category.ToUpper();
    return $"{prefix}-{id:D4}";
}
}
}

```

Code Explanation

- The controller **does not accept** sensitive data like CostPrice or SupplierDiscount from the client.
- It **calculates** them internally based on business rules.
- The **SKU** is dynamically generated using the category name and product ID — e.g., "ELEC-0005".

Advantages

- The system remains secure.
- Clients only see relevant, safe data.
- Business rules are enforced at the API level.

Difference Between [JsonIgnore] Attribute and DTOs in ASP.NET Core Web API

Both [JsonIgnore] and **DTOs (Data Transfer Objects)** are used to **control what data is exposed to or accepted from clients**, but they differ significantly in **scope, flexibility, and purpose**.

[JsonIgnore] Attribute

The [JsonIgnore] attribute (from System.Text.Json.Serialization) tells the ASP.NET Core serialization system to **ignore a specific property** during JSON serialization (output) or deserialization (input).

In other words:

- When sending data **to the client**, the ignored property won't appear in the JSON response.

- When receiving data **from the client**, any value for that property will be ignored.

Use it when:

- You have **small or simple models** and want to hide only a few fields.
- You want **quick control** over which properties are exposed.
- The hidden fields are always meant to be excluded in every API operation (GET, POST, PUT, etc.).
- You want to **avoid creating multiple DTO classes** for lightweight APIs.

Limitations of [JsonIgnore]

- It's **too rigid**; the property is hidden **globally** (you can't expose it in one endpoint but hide it in another).
- Not suitable for **large APIs** where input/output shapes vary per operation (e.g., different data in Create vs. Update).
- It is difficult to maintain when your domain model changes. Every API gets affected.

Data Transfer Objects (DTOs)

DTOs are **separate classes** explicitly created to define what data is:

- **Received from the client (input DTO)**, or
- **Returned to the client (output DTO)**.

DTOs act as a **communication layer** between your API and your domain models, allowing you to shape data **independently** of your database entities.

Use DTOs when:

- You have **medium-to-large APIs** or multiple endpoints for the same entity.
- You want **different shapes** of data for different operations (e.g., Create vs. Update vs. Get).
- You need to **rename**, **combine**, or **transform** fields before sending them to the client.
- You want to **protect your domain model** from direct client exposure.
- You're following **Clean Architecture** or **DDD (Domain-Driven Design)**, and DTOs are a best practice here.

Limitations of DTOs

- Requires **extra classes and mapping code** (manual or via tools like AutoMapper).
- Slightly more **code overhead** for small projects.
- May feel like using more words than necessary for very simple APIs.

Summary:

In ASP.NET Core Web API, **excluding properties from model binding** is essential for secure and maintainable API design.

- Use [JsonIgnore] to permanently exclude fields globally.
- Use **DTOs** to gain fine-grained control over what data clients can send or receive.
- Implement **internal business logic** (like SKU generation and cost computation) inside controllers or services — not in client-supplied data.

This approach ensures that sensitive or system-maintained data stays protected, while clients interact only with safe, clearly defined API contracts.

In the next article, I will discuss how to implement server-side [Validation using Data Annotations in ASP.NET Core Web API](#) with Examples. In this article, I try to explain **how to include and Exclude Properties from Model Binding in ASP.NET Core Web API** with Examples, and I hope you enjoy this article, “How to Include and Exclude Properties from Model Binding in ASP.NET Core Web API.”



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

1 thought on “Exclude Properties from Model Binding in ASP.NET Core Web API”



SAMEERA

JUNE 3, 2025 AT 1:50 PM

Thank u sir

[Reply](#)

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information